

OpenCV 영상 전송 URDF 모델링과 rVIZ 시각화

Junha Lee

Architecture and Compiler for Embedded System LAB.

School of Electronics Engineering, KNU, KOREA

2026-06-30



목차

- OpenCV 영상 전송
- URDF 모델링
- Rviz 시각화
- 아두이노 프로세스 문제

Open CV

- cv_bridge
OpenCV와 ROS2 Image 메시지 상호 변환
- Sensor_msgs
이미지 표준 메시지 정의
- Image_transport

Image_publisher_node.py

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import Header
from cv_bridge import CvBridge, CvBridgeError

from rclpy.qos import QoSProfile
from rclpy.qos import QoSHistoryPolicy
from rclpy.qos import QoSDurabilityPolicy
from rclpy.qos import QoSReliabilityPolicy

import sys
import cv2
import os
```

```
# Publish할 토픽 이름
PUB_TOPIC_NAME = 'image_raw'

# 데이터 입력 소스: 'camera', 'image', 또는 'video' 중 택1하여 입력
DATA_SOURCE = 'camera'

# 카메라(웹캠) 장치 번호 (ls /dev/video* 명령을 터미널 창에 입력하여 확인)
CAM_NUM = 0

# 이미지 데이터가 들어있는 디렉토리의 경로를 입력
IMAGE_DIRECTORY_PATH = 'src/camera_perception_pkg/camera_perception_pkg'

# 비디오 데이터 파일의 경로를 입력
VIDEO_FILE_PATH = 'src/camera_perception_pkg/camera_perception_pkg/lib'

# 화면에 publish하는 이미지를 띄울것인지 여부: True, 또는 False 중 택1하여 입력
SHOW_IMAGE = True

# 이미지 발행 주기 (초) - 소수점 필요 (int형은 반영되지 않음)
TIMER = 0.03
```

소스 선택

- camera / video / image 세가지 입력 소스를
파라미터로 전환

주기발행

- 0.03초 >> 약 33fps 로 프레임을 반복해서
발행

Image_publisher_node.py

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import Header
from cv_bridge import CvBridge, CvBridgeError

from rclpy.qos import QoSProfile
from rclpy.qos import QoSHistoryPolicy
from rclpy.qos import QoSDurabilityPolicy
from rclpy.qos import QoSReliabilityPolicy

import sys
import cv2
import os
```

```
# Publish할 토픽 이름
PUB_TOPIC_NAME = 'image_raw'

# 데이터 입력 소스: 'camera', 'image', 또는 'video' 중 택1하여 입력
DATA_SOURCE = 'camera'

# 카메라(웹캠) 장치 번호 (ls /dev/video* 명령을 터미널 창에 입력하여 확인)
CAM_NUM = 0

# 이미지 데이터가 들어있는 디렉토리의 경로를 입력
IMAGE_DIRECTORY_PATH = 'src/camera_perception_pkg/camera_perception_pkg'

# 비디오 데이터 파일의 경로를 입력
VIDEO_FILE_PATH = 'src/camera_perception_pkg/camera_perception_pkg/lib'

# 화면에 publish하는 이미지를 띄울것인지 여부: True, 또는 False 중 택1하여 입력
SHOW_IMAGE = True

# 이미지 발행 주기 (초) - 소수점 필요 (int형은 반영되지 않음)
TIMER = 0.03
```

- Cv2.VideoCapture
- image_publisher_node
cv2_to_imgmsg >> publish
- /image_raw 토픽
sensor_msgs/Image
- Subscriber node
imgmsg_to_cv2 >> OpenCV 처리

URDF

- Link + Joint 트리 구조
로봇의 형상, 관성, 관절 관계를 XML로 작성
- Link
로봇의 실제 부품(몸체, 바퀴 등)
- Joint
두 Link를 연결하는 관절 등을 정리.

URDF

```
<link name="body_link">
  <visual>
    <!-- origin: 이 Link의 시각적 형상이 표시될 위치와 회전 -->
    <!-- xyz: X, Y, Z 방향 이동 (단위: 미터) -->
    <!-- rpy: Roll(X축), Pitch(Y축), Yaw(Z축) 회전 (단위: 라디안) -->
    <origin rpy="0 0 0" xyz="0 0 0"/>

    <!-- geometry: 3D 형상 정의 -->
    <geometry>
      <!-- box: 직육면체 형상 -->
      <!-- size: X(길이) Y(폭) Z(높이), 단위: 미터 -->
      <!-- 이 몸체는 1m(길이) × 0.6m(폭) × 0.3m(높이) -->
      <box size="1 0.6 0.3"/>
    </geometry>

    <!-- material: 색상 및 재질 정의 -->
    <material name="red">
      <!-- rgba: Red Green Blue Alpha (투명도) -->
      <!-- 각 값은 0~1 사이, 여기서는 순수 빨간색(불투명) -->
      <color rgba="1 0 0 1"/>
    </material>
  </visual>
</link>
```

```
<link name="wheel1_link">
  <visual>
    <!-- 원통(Cylinder)은 기본적으로 Z축 방향으로 생성됨 -->
    <!-- 바퀴는 옆으로 누워야 하므로 X축 기준 90도(1.570795 라디안) 회전 -->
    <origin rpy="1.570795 0 0" xyz="0 0 0"/>

    <geometry>
      <!-- cylinder: 원통 형상 -->
      <!-- length: 원통의 길이(두께) 0.1m -->
      <!-- radius: 원통의 반지름 0.15m (직경 0.3m) -->
      <cylinder length="0.1" radius="0.15"/>
    </geometry>

    <material name="yellow">
      <!-- 노란색 바퀴 -->
      <color rgba="1 1 0 1"/>
    </material>
  </visual>
</link>
```

RViz

- Robot_state_publisher
URDF >> /joints_states 구독 >> /tf 발행
- Joint_state_publisher_gui
슬라이더로 관절 각도를 /joints_states로 발행
- Rviz2
/tf, /robot_description을 받아 3D 렌더링
- .rviz 파일은 GUI로 화면을 구성한 뒤
File >> Save Config As 로 Rviz가 자동생성한다

Serial_sender_node

- Ros2 launch 실행 시 정상 실행은 되지만 종료 시 아두이노 연결 노드에 대한 프로세스를 직접 kill해야하는 문제
- `ser = serial.Serial(PORT, 115200, timeout=1)`
시리얼 객체가 모듈 전역에서 열림
- `self.ser = serial.Serial(PORT, 115200, timeout=1, write_timeout=1)`
시리얼 연결을 SerialSenderNode.__init__ 내부로 옮김

Serial_sender_node

```
try:
    rclpy.spin(node)

except (KeyboardInterrupt, SystemExit):
    pass

finally:
    print("\n\nshutdown\n\n")
    node.send_stop_command()
    node.ser.close()
    print('closed')
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()
```

- 종료 처리가 KeyboardInterrupt에만 의존
- 정지 명령 전송 >> 포트 닫힘 >> destroy_node >> shutdown

Motion_planner_node

```
# 경로 각도를 조향 단계로 반영 비례 배율 주 MAX_STEER로 곱함
steer = target_slope / STEER_AT_FULL_LOCK_DEG * MAX_STEER
steer = max(-MAX_STEER, min(MAX_STEER, steer))
self.steering_command = int(round(steer))
```

- 조향이 항상 양측 7로 고정
>> 선형 비례 조향으로 변경
각도가 커짐에 따라 조향각 변경

경로 각도	계산	조향 명령
0°	$0/30 \times 7 = 0$	0
5°	$5/30 \times 7 = 1.17$	1
10°	$10/30 \times 7 = 2.33$	2
15°	$15/30 \times 7 = 3.5$	4
20°	$20/30 \times 7 = 4.67$	5
25°	$25/30 \times 7 = 5.83$	6
30°	$30/30 \times 7 = 7.0$	7
40°	$40/30 \times 7 = 9.33$	7 (잘림)
-10°	$-10/30 \times 7 = -2.33$	-2
-30°	$-30/30 \times 7 = -7$	-7

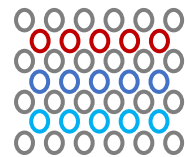
Motion_planner_node

```
def timer_callback(self):  
    # 시작 직후 직진 정렬 단계: 정지 상태로 steering=0만 보내 바퀴를 중앙 정렬  
    elapsed = (self.get_clock().now() - self.start_time).nanoseconds / 1e9  
    if elapsed < STARTUP_STRAIGHT_SEC:  
        self.steering_command = 0  
        self.left_speed_command = 0  
        self.right_speed_command = 0  
        self.publish_motion_command()  
    return
```

- 시작 시 바퀴가 직진으로 정렬되지 않음
 >> 시작시
 STARTUP_STRAIGHT_SEC를 통해 바퀴 정렬시간 가짐(3초로 설정)

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)